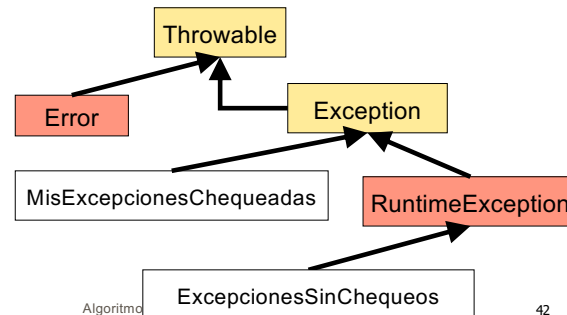


Jerarquía de Clases de Excepciones

Java representa las excepciones y errores como objetos instancia de clases de excepciones



Algoritmo

42

●42

Levantamiento de Excepciones

● Estructura general

`throw ObjetoDeTipoExcepcion;`

● Aborta lo que se ejecuta y se cede el control a donde se hace el tratamiento:

- throw está dentro de un manejador de excepciones que trata ese tipo de excepciones
 - La tratamos en el manejador
- throw no está dentro de un manejador, pero el método levanta ese tipo de excepciones

modificadores Tipo nombre

(Lista parametros) throws ClaseExcep1, ClaseExcep2...

- Se aborta el método y esa excepción se levanta donde se hizo la llamada

Algoritmos y Estructuras de datos DIT-UPM

43

●43

Manejadores de Excepciones

- Estructura general



```
try { cuerpo-try
} catch (ClaseExcepcion1 ref1) { tratamiento ClaseExcepcion1
} catch (ClaseExcepcion2 ref2) { tratamiento ClaseExcepcion2
} finally { cuerpo finally
}
```

- cuerpo-try representa la ejecución en condiciones normales
- catch indica como tratar cada tipo de excepción en caso de que un throw o una llamada a método la levante
 - Cuando una excepción salta se busca (en el orden de la declaración) un catch que pueda tratarlo (el tipo de la excepción es compatible)
 - Si no hay ninguno, la excepción sigue levantada y la debe tratar aquello que incluya la sentencia try
- finally es opcional e incluye aquello que debemos ejecutar tanto si se produce, como si no, una excepción

Algoritmos y Estructuras de datos DIT-UPM

44

●44

Contenidos

1. Repaso de herramientas

2. Repaso de programación

2.1 Tipos simples de datos y expresiones

2.2 Sentencias

2.3 Clases y Objetos

2.4 Interfaces y Herencia

2.5 Excepciones

2.6 Bibliotecas

Algoritmos y Estructuras de datos DIT-UPM

45

●45

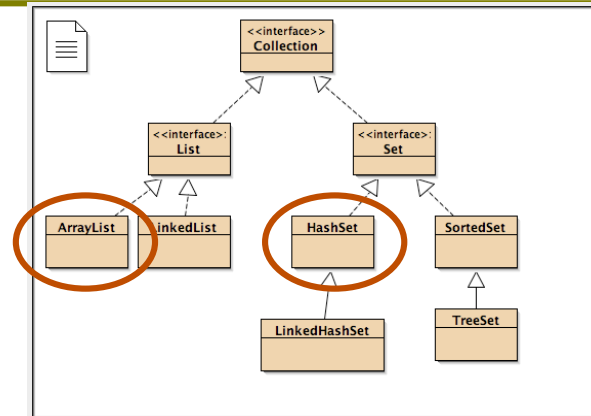
Colecciones

- Los programas manejan gran cantidad de datos del mismo tipo
- Los **arrays** permiten hacerlo, pero **tienen tamaños fijos**
- Las **colecciones** son clases predefinidas que **permiten almacenar datos de tipo compatible**
 - El **tamaño puede variar** –no predeterminado
 - Se pueden manejar estos datos de formas diferentes (Lista, Conjunto, Mapa)
 - **Trabajan sólo con objetos**
 - Importante: Clases envoltorio para tipos primitivos

Algoritmos y Estructuras de datos DIT-UPM

●46

Jerarquías de colecciones

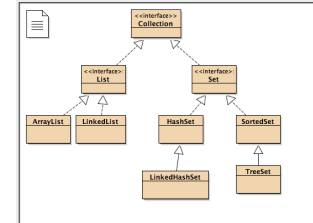


Algoritmos y Estructuras de datos DIT-UPM

●47

Métodos de la interfaz Collection<E>

- **boolean add(E elemento)**
- void clear()
- **boolean contains(E elemento)**
- boolean equals (Object x)
- **boolean isEmpty()**
- Iterator<E> iterator()
- **boolean remove(E elemento)**
- **int size()**
- **Object[] toArray()**
- ...



Algoritmos y Estructuras de datos DIT-UPM

●48

interface List<E>

- boolean add(E elemento)
 - void add(int posicion, E elemento)
 - void clear()
 - boolean contains(E elemento)
 - boolean equals(Object x)
 - E get(int posicion)
 - int indexOf(E elemento)
 - boolean isEmpty()
 - Iterator<E> iterator()
 - E remove(int posicion)
 - boolean remove(E elemento)
 - E set(int posicion, E elemento)
 - int size()
 - Object[] toArray()
- [En rojo: pueden lanzar excepciones]**

Algoritmos y Estructuras de datos DIT-UPM

●49

Implementaciones de List<E>

- ArrayList<E>
 - array dinámico
 - Usa poca memoria
 - Recorridos rápidos, inserción y borrado lento
- LinkedList<E>
 - lista encadenada
 - Usa memoria extra
 - Recorrido lineal rápido, otros lentos
 - inserciones y eliminaciones internas rápidas

Algoritmos y Estructuras de datos DIT-UPM

●50

```
List<String> nombres = new ArrayList<>();
```

interface Set<E>

- boolean add(E elemento)
- void clear()
- boolean contains(E elemento)
- boolean equals(Object x)
- boolean isEmpty()
- Iterator<E> iterator()
- boolean remove(E elemento)
- int size()

[En rojo: pueden lanzar excepciones]

Algoritmos y Estructuras de datos DIT-UPM

●51

Implementaciones de Set<E>

- class HashSet<E> implements Set<E>
 - económica en tiempo y memoria
- class TreeSet<E> implements Set<E>
 - más lenta y voluminosa
 - cuando se recorre los elementos salen ordenados

Algoritmos y Estructuras de datos DIT-UPM

●52

interface Map<K, V>

- void clear()
- boolean containsKey(Object clave)
- boolean containsValue(Object valor)
- boolean equals(Object x)
- V get(Object clave)
- boolean isEmpty()
- Set<K> keySet()
- V put(K clave, V value)
- V remove(Object clave)
- int size()
- Collection<V> values()

[En rojo: pueden lanzar excepciones]

Algoritmos y Estructuras de datos DIT-UPM

●53

Implementaciones de Map<K, V>

- HashMap
 - económica
- LinkedHashMap
 - respeta el orden de inserción
 - voluminosa
- TreeMap
 - claves ordenadas
 - lenta y voluminosa

Algoritmos y Estructuras de datos DIT-UPM

●54

Pruebas automáticas

- JUnit es una biblioteca de clases para ayudar a programar y ejecutar pruebas unitarias
- Para probar una clase hay que crear otra clase de prueba
 - Para probar Punto, la clase de prueba es TestPunto
- Por cada método de la clase bajo pruebas se escriben varios métodos en la clase de prueba

Fundamentos de
Programación DIT-
UPM

55

●55

Prueba con JUnit 5.x

- **Importar clases** JUnit

```
import org.junit.jupiter.api.*;
import static org.junit.Assert.*;
(import static: para referenciar métodos/atributos sin citar la clase)
```

- **Declarar clase de prueba (normal)**

```
public class MiClaseTest {
```

- **Declarar variables para prueba:**

```
MiClase c;...
```

- **Uso de anotaciones** (metadatos) @...

- Instrucciones para el compilador y otras herramientas
- Igual que para JavaDoc: (@author, @param, ...)

56

Fundamentos de
Programación DIT-
UPM

●56

Anotación de métodos

- **@Test**: método que representa un caso de prueba
 - Comprueba si el resultado esperado es igual al real
 - Métodos Assert: assertTrue, assertFalse, assertEquals, assertNull, assertNotNull, assertEquals, assertEquals
- **@Before/@After**: métodos que deben ejecutarse antes/después de cualquier caso de prueba
- **@BeforeAll/@AfterAll**: se ejecutan **una vez**, **antes/después** de todos los test de la clase
 - Puede haber varios métodos etiquetados con @Before y @After
 - Solo un @BeforeAll y un @AfterAll

57

Fundamentos de
Programación DIT-
UPM

●57